Field Hours

CSCE 313 Introduction to Computer Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Pointers and bytes

A pointer holds an address. The type says how many bytes a read through it means. An int pointer steps four bytes on many machines. A char pointer steps one. Adding one to a pointer adds the size of the pointed-to type, not the number one.

A buffer of n bytes does not contain n integers. Divide by the element size before you index. Writing past the end is the bug CSCE 465 will name. Here it is just a wrong address.

Four bytes, two readings.

As four chars

As one int

The address of the first byte is both

An array of two ints

1. The array starts at address 1000. The first int occupies 1000 through 1003.

2. The second starts at 1004. Pointer plus one, typed as int, is 1004.

3. The same pointer typed as char plus one is 1001, the next byte, which is the middle of the first int.

Problem 1. You allocate 8 bytes and store three ints of 4 bytes. What happens?

Answer. The third int writes past the allocation, into whatever follows. The language will not stop you in C. The following value changes, or the program faults if the next page is unmapped.

Problem 2. Why does pointer arithmetic depend on the type?

Answer. The compiler scales by the element size so that p + 1 means the next element, not the next byte. Cast the pointer and the step changes. The bits in memory do not.

Lecture 2 · 50 minutes

A process

A process is a program in execution: an address space, registers, and the kernel's record of open files. Fork duplicates that record. The child is a copy. Exec replaces the child's program. A shell does both: fork, then exec in the child, while the parent waits.

The two processes do not share memory after a normal fork, except for pages the kernel marks copy-on-write. A write then gets a private copy. They do share the file offset if they inherited the same open file. That surprise is

worth one experiment.

A shell running a command.

Fork -> Child execs the program -> Parent waits -> The child exits and the parent continues

One fork

1. Fork returns twice. The parent receives the child's id. The child receives 0.

2. Both continue after the call. The child calls exec. If exec works, the child does not return.

3. If the parent does not wait, it can finish and leave the child to be reaped later. The zombie is the child the parent has not waited for.

Problem 1. Fork returns 0 in the child and a positive id in the parent. Why not the same number?

Answer. The parent needs the id to wait or signal. The child already knows it is the child. Returning the parent's id to the child is unnecessary, and returning 0 is the convention the child checks.

Problem 2. Both parent and child write to the same inherited file, with no seek of their own. What do you expect?

Answer. The writes share one offset. They interleave in the file, and neither has a private position. Dup'd descriptors are not independent cursors.

Lecture 3 · 50 minutes

The address space

The process sees a private range of addresses: code, globals, a heap that grows on allocation, and a stack that grows on calls. Those are virtual addresses. The kernel's page tables make them physical. Another process can use the same numbers and mean different memory.

A null pointer is address 0, which the kernel leaves unmapped so a mistaken use faults instead of silently changing data. That fault is a feature.

A process, from high addresses to low.

Stack

Hole

Heap

Globals and code

Two allocations

1. Malloc of 16 bytes extends the heap, or takes a free block already there.

2. A local array of 16 bytes sits on the stack and disappears when the function returns.

3. Returning a pointer to that local is a dangling pointer. The next call reuses the slot.

Problem 1. Why is a stack allocation faster than malloc, and when is it the wrong tool?

Answer. It is a move of the stack pointer, reclaimed on return. It is wrong when the value must outlive the function, or when the size is huge or unknown for the thread's stack.

Problem 2. Process A writes address 1000. Does process B's address 1000 change?

Answer. No, unless they deliberately share a page. The numbers are virtual and the tables differ.

Lecture 4 · 50 minutes

Threads and a race

A thread is a second flow of control in the same address space. It has its own stack and shares the heap and globals. Sharing is why threads are cheap, and why they race. A race is two threads using the same location, at least one writing, and no order that the program enforces.

A lock makes a region exclusive. The region should be the update, not the whole program. A lock you forget on one path is still a race. CSCE 611 will add the proof habits. This course asks you to spot the unprotected increment.

One counter, two threads.

Without a lock

Both read 0

Both write 1

One increment is lost

With a lock

One holds the lock

The other waits

The counter ends at 2

The lost increment

1. x is 0. Thread A reads 0. Thread B reads 0.

2. A writes 1. B writes 1. The second increment never saw the first.

3. A lock around the read and the write forces B to read 1 and write 2.

Problem 1. Two threads only read a shared table that nobody writes after startup. Is there a race?

Answer. No. A race needs a write, or a synchronization bug the language defines as one. Concurrent readers of stable data are the normal case.

Problem 2. You lock on the way in and return early on an error without unlocking. What happens to the other thread?

Answer. It waits forever. Every path out of the region unlocks, including the error returns. A structure that unlocks on scope exit is how you stop forgetting.

Lecture 5 · 50 minutes

Files

A file is a name in a directory and bytes on disk. The directory maps the name to an inode. The inode holds the size and the block map. Open returns a small integer, the descriptor, that points at the kernel's open-file record. Read and write use the descriptor, not the name.

Close releases the descriptor. Forgetting it leaks a slot, and a long-running server eventually cannot open anything. Unlink removes a name. The bytes stay until the last name and the last descriptor are gone.

Open, then read.

1. Look up the name

2. Check permission

3. Return a descriptor

4. Read uses the descriptor

Two names, one file

1. Link creates a second name for the same inode. The link count is 2.

2. Unlink the first name. The count is 1. Reads through the second name still see the bytes.

3. Unlink the second name and close every descriptor. The blocks can be freed.

Problem 1. You check the name, then open it later. What can change in between?

Answer. The name can be replaced with a different file, or a link to one you did not check. The check and the open have to be the same lookup. This is the time-of-check bug the security course will repeat.

Problem 2. Read returns 20 when you asked for 100. Is that an error?

Answer. Not always. A short read is normal at end of file, or from a pipe, or after a signal. Loop until you have 100, or until a real zero or error. Treating a short read as failure drops data.

Lecture 6 · 50 minutes

What the graduate OS course will assume

You can now say what a process is, why threads share, what a descriptor names, and why address 1000 in two processes is not one place. CSCE 611 starts at scheduling policy, page replacement, and crash consistency of a file system. It will not rederive fork.

Write one of those four from memory before you start 611. If you cannot, this prerequisite is not finished.

The four sentences.

A process is an address space plus kernel state

Threads share that space and can race

A descriptor is not a name

Virtual addresses are per process

A question you should answer cold

1. What does fork return in each process?

2. The parent gets the child id. The child gets 0. Both continue. The child usually execs.

3. If that sentence is not immediate, reread the process lecture before the graduate course.

Problem 1. Which course is the prerequisite for CSCE 611, and which fact from it does 611 not reteach?

Answer. CSCE 313. Fork, the address space, and the descriptor. 611 teaches policy and mechanism on top of those.

Problem 2. Name one bug from this course that CSCE 465 will call an attack.

Answer. Writing past a buffer onto a saved return address, or a race on a check. The layout lecture in 312 and the pointer lecture here are the same memory.